



Figure 1: Web application testing tools

Splinter

Splinter is a Web application testing tool with easy-to-use functions for most of the frequently performed tasks. What makes Splinter stand out is the ease with which it allows the automation of tasks. Splinter was developed as an easily usable abstraction layer on top of the available automation tools such as Selenium, PhantomJS, etc. The following code sequence, to automatically fill a value into a text box, will provide a glimpse of how simple it is to perform tasks using Splinter:

```
browser.fill('search', 'open source for you')
```

To perform the same thing in Selenium, the code is shown below:

```
my_node = browser.find_element.by_name('search')
my_node.send_keys('open source for you')
```

The features of Splinter are illustrated in Figure 2.

Splinter installation

As Splinter is Python based, using it requires Python to be installed in your system. Splinter is supported by Python version 2.7+. If your system has Python installed in it, then Splinter installation can be carried out with the help of 'pip' as shown below:

```
$ pip install splinter
```

If you are behind a proxy server, then the corresponding proxy settings need to be made before executing the aforementioned 'pip' command.

Supported Web drivers

Splinter has support for all the major Web drivers listed below:

- Chrome Web driver
- Firefox Web driver
- PhantomJS Web driver
- Remote Web driver
- Zoptest browser

As the support spans a spectrum of drivers, Splinter can be utilised in various scenarios that require any of the above-mentioned drivers.

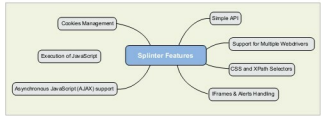


Figure 2: Splinter's features

Splinter demo with the OSFY page

To get familiar with Splinter, let's write a few lines of Python code which automate the following tasks:

1. Open the Firefox browser
2. Navigate to <http://opensourceforu.com>
3. Enter the search query in the provided search input box on the Web page
4. Click on the *Submit* button (the magnifier icon, in this case)
5. Check whether the resultant page has a specific keyword

As you will infer from the following code, first an instance of the browser is built using Splinter, and then...

```
from splinter import Browser
with Browser() as browser:
    # Step 1 & 2
    url = "http://opensourceforu.com/"
    browser.visit(url)
    # Step 3
    browser.fill('s', 'zotero')
    # Step 4
    button = browser.find_by_tag('button').first
    button.click()
    # Step 5
    if browser.is_text_present('Research'):
        print("Yes, the Article Found")
    else:
        print("No, it wasn't found")
    raw_input();
```

...the specified OSFY page is loaded, as follows:

```
from splinter import Browser
with Browser() as browser:
    url = "http://opensourceforu.com/"
    browser.visit(url)
```

Filling the search query in this case is carried out very quickly by calling the fill method of the browser object with the name of the textbox ('s' in this case) and query as parameters:

```
browser.fill('s', 'zotero')
```

Then, finding the *Submit* button would have been very